

Movie Catalogue Service

Çağla Nur Yurdasal - 34367

1. Tools Used

Claude (Anthropic): Sonnet 4.6

2. How AI Was Used in This Project

For the Movie Catalogue Service I used Claude throughout the development process as a guided tutor rather than a code generator. The core agreement I set with myself early on was: I write every line of code myself, and Claude explains the logic, points out mistakes, and answers "why" questions: it doesn't write code for me to copy-paste.

a. Conceptual Explanation

Example Prompt 1: Understanding before implementing

Before writing models.py, I asked Claude to explain what a Pydantic model is conceptually, without writing any code. I then wrote my own Movie class from scratch, and Claude reviewed it afterward: flagging that I'd made 'id' and 'year' either too strict or not strict enough, and that I was missing the Config class for Pydantic aliasing. I also asked how to verify optional fields in the real dataset:

Prompt

what is a pydantic model and why do I need it

Claude response

A Pydantic model is a Python class that defines what your data should look like. You list the fields and their types, and Pydantic automatically checks that any data coming in matches those rules.

For example your Movie model says: title must be a string, year must be an integer, genres must be a list of strings. If someone sends a request with year as "hello" instead of a number, Pydantic rejects it automatically and returns a clear error message, without you writing any validation code yourself.

You need it in your project for two reasons:

First, validation. Your API receives data from the Android app and other services. Without a model, you have no guarantee the data is in the right format, and your code could crash in unexpected ways when it tries to use a field that is missing or the wrong type.

Second, documentation. FastAPI reads your Pydantic models and uses them to generate the Swagger UI at /docs automatically. That is how the docs know what fields each endpoint expects and returns. Without models, the docs would be empty and your teammates would have no way to see the shape of your data without reading your code directly.

Prompt

how can I check which fields are optional when I look at the dataset and the collection?

Claude response

Open MongoDB Compass and connect to your cluster. Then:

1. Click on sample_mflix → movies
2. Look at several different movie documents – click through at least 10-15 of them
3. For each field, ask yourself: does every single movie have this field, or do some movies not have it?

If a field is missing in even one document, it should be Optional in your model.

You can also do it faster with a query. In Compass, click "Open MongoDB Shell" at the bottom and run these one by one:

```
// count how many movies are missing a plot
db.movies.countDocuments({ plot: { $exists: false } })
// count how many movies are missing a poster
db.movies.countDocuments({ poster: { $exists: false } })
// count how many movies are missing a year
db.movies.countDocuments({ year: { $exists: false } })
// count how many movies are missing rated
db.movies.countDocuments({ rated: { $exists: false } })
// count how many movies are missing runtime
db.movies.countDocuments({ runtime: { $exists: false } })
```

If the count is 0 – the field exists in every document, it could be required. If the count is greater than 0 – some documents are missing it, make it Optional.

I'll tell you now that title is the only one that's truly required in every document. Everything else will have missing values. But run the queries yourself so you can explain it to the graders – "I checked the dataset and found X movies were missing this field, so I made it optional."

I went back to MongoDB Compass myself to verify which fields were actually optional by running count queries, rather than taking Claude's word for it.

b. Debugging

Example Prompt 2: MongoDB authentication error

When I hit the following error trying to connect to MongoDB Atlas:

Prompt

File

```
"/Users/cagla/group_1_05/movie-catalogue-service/.venv/lib
/python3.13/site-packages/pymongo/helpers_shared.py", line
284, in _check_command_response
    raise OperationFailure(errmsg, code, response,
max_wire_version)
```

```
pymongo.errors.OperationFailure: bad auth : Authentication failed., full error: {'ok': 0, 'errmsg': 'bad auth : Authentication failed.', 'code': 8000, 'codeName': 'AtlasError'}
```

I'm getting this error when running my test_connection.py script

Claude response

Good news – this is a different error than before. The URI is being read correctly (no more localhost error), but the password is wrong. The key line is:

```
OperationFailure: bad auth : Authentication failed.
```

Most likely cause: the autogenerated password contains special characters (like @, /, !, #) that break the URI format. These characters have special meaning in a URL and need to be escaped.

Fix it – go back to Atlas and reset the password:

1. Atlas → Security → Database Access → click Edit on your movieapp user
2. Click "Edit Password" → this time type a simple password manually – only use letters and numbers, no special characters. Something like Movieapp2026
3. Click Update User

Then update your .env file:

```
MONGO_URI=mongodb+srv://movieapp:Movieapp2026@cluster0.xxxx.mongodb.net/
```

Run the test script again and it should print the movie count.

Alternatively if you want to keep the autogenerated password, you can URL-encode the special characters – but resetting to a simple one is much easier and fine for a student project.

Example Prompt 3: Android Studio / Gradle JDK error

When trying to run the Android frontend to test the full system:

Prompt

When I try to run the application on Android Studio I get this error:

```
FAILURE: Build failed with an exception.
```

```
Execution failed for task ':app:compileDebugJavaWithJavac'.
```

```
jlink executable  
/Users/cagla/.vscode/extensions/redhat.java-1.54.0-darwin-arm64
```

```
/jre/21.0.10-macosx-aarch64/bin/jlink does not exist.
```

Claude response

```
Android Studio is using the JDK from VS Code's Java
extension instead of its own

bundled JDK. Go to Settings -> Build, Execution,
Deployment -> Build Tools ->

Gradle and change the Gradle JDK to 'Embedded JDK' or the
Android Studio

bundled path: /Applications/Android
Studio.app/Contents/jbr/Contents/Home
```

This was a local environment problem unrelated to my service code. Claude helped diagnose it quickly without me spending hours searching the error message manually.

c. Code Review of My Own Implementation

Example Prompt 4: Structuring the Pydantic model

Before writing models.py I asked about the structure without asking for the code:

Prompt

```
should I create a class for every field or one class for
the whole movie? without writing the code, help me build
the structure of the code
```

Claude response

```
No, one class for the whole movie. Each field is just a
line inside that class.

Think of it like a form. The form itself is the class, and
each field on the form is a line inside it. You don't
create a separate form for each field.

class Movie(BaseModel):
    field1: type
    field2: Optional[type] = None
    ...

Now try writing it yourself and let me know.
```

Example Prompt 5: Pagination implementation review

I wrote the list_movies endpoint and pasted it for review:

Prompt

I want to use limit-offset approach. here is what my code looks like:

```
@router.get('/')
def list_movies(page: int = 1, limit: int = 20):
    offset = (page - 1) * limit
    movies =
movie_collection.find().skip(offset).limit(limit)
    return [serialize(m) for m in movies]
```

Claude response

Good, the rename is clean and consistent. One thing is missing:

limit needs a maximum constraint using Query()

Right now someone could call GET /movies/?limit=50000 and your service would try to fetch 50,000 movies at once, which could crash everything. You need to tell FastAPI to reject anything over the max limit.

Example Prompt 6: Filter endpoint bug catch

Claude caught a subtle bug in my filter endpoint where I'd used truthy checking instead of None-checking:

Prompt

```
if year_min:
    year_condition['$gte'] = year_min
if year_max:
    year_condition['$lte'] = year_max
```

Claude response

if year_min and if year_max have a bug when year is 0.

Using 'if year_min:' checks if the value is truthy. In Python, 0 is falsy,

so if someone passes year_min=0 it would be skipped. The correct way to check

if a parameter was actually provided is:

```
if year_min is not None:
```

```
if year_max is not None:
```

I fixed this myself after understanding why it was wrong.

3. Reflection

Using Claude as a "no-code-given" tutor changed how I approached this project. Instead of asking it to write code for me, I set a rule early on: ask for the concept first, write the code myself, then ask for review. This meant that by the time I got to the technical discussion, I could explain every decision. Why I used \$in for array fields but direct equality for rated, why the text index already existed in sample_mflix and creating a new one caused a conflict, or why MongoClient is lazy and does not connect at import time.

The biggest advantage was fixing environment issues quickly. Docker errors, Gradle/JDK conflicts, MongoDB authentication failures, and git merge conflicts would have cost me a lot of time. Having Claude read real error messages and suggest fixes meant I could solve problems in minutes instead of hours.

The main limitation was that Claude sometimes gave textbook answers without knowing my specific data. For example, it suggested creating a text index on title and plot, but that index already existed in sample_mflix and caused an error. I learned to always verify in MongoDB Compass instead of trusting what Claude assumed about the dataset.

Overall, AI was most useful as a learning tool and a debugging assistant, not as a replacement for understanding the code myself.